

Kaleb Hannan
COS_442
Final Project Writeup
4/23/2025

Project Overview:

My project is called the Baseball Stats System. This application lets users enter baseball players and teams and store their data in a database. The user can then retrieve that data and display it in the application. The application has three basic parts: the homepage, the player view and create, and the team view and create. The home page allows the user to add a new player or team, or search for a team or player stored in the Redis database. The player side of the application creates the functionality for adding and retrieving new players to the database and rendering the HTML to display the information. The player's side of the application is made up of four containers: the player-report, player-bio, player-batting, and player-fielding-pitching. The team's side of the application has all the functionality for storing and retrieving information about the teams stored in the database. The team's side of the application comprises two containers, the team-report and the team-info containers.

How to Use:

To use this application, first unzip the FinalProject zip file. After unzipping the application, add the application to a Jetstream instance. Now, in the Jetstream instance, 'cd' to the FinalProject folder and when in that directory use the 'docker compose up --build -d' command.

Now that the application is running on Docker in a Jetstream instance, in a browser, go to the public IP of the instance base URL, which is (http: "PublicIP":80/). When you go here, it will bring you to the application's home page as seen in Figure 3 (Home Page) below.

The application has four functions that can all be accessed with the home page: add a new player, add a new team, search for a player, or search for a team. To add a new player, click on the "Add New Player" button on the home page, which will bring you to the form to add a new player, as

seen in Figure 5 (Add New Player Bio Form). To add a new team, click on the “Add New Team” button on the home page, which will bring you to the form to add a new team as seen in Figure 14 (Add New Team Form).

To search for a player, type the player's name into the search bar under the “Search for a Player Here ” section. If the player is found in the Database, it will redirect you to the player report page with the information for the player that you searched for, as seen in Figure 4 (Player Report Page). If the player is not found, it will render an error message as seen in Figure 1.

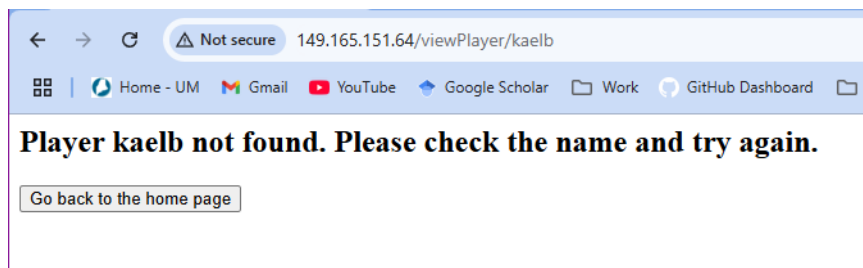


Figure 1: Player Search Error Page

To search for a team, type the team’s name into the search bar under the “Search for a Team Here” section. If the team is in the Database, it will redirect you to the team report page with the information for the team you searched for, as seen in Figure 13 (Team Report Page). If it is not found in the database, then it will return an error message as seen in Figure 2

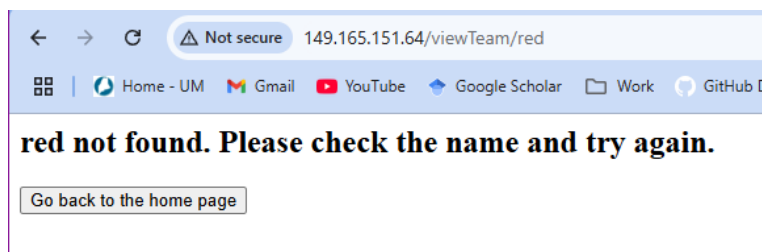


Figure 2: Team Search Error Page

After you are finished with the application, you can shut it down using “docker compose down.”

For more information about how each service works, see the “Description of Each Service and Its Routes” section below.

Description of Each Service and Its Routes:

home-page:

The home page service acts as the application's landing page. It can also direct users to other services and functions of the application, like adding new teams and players and searching for players.

`@app.route('/')`

def home():

This route renders the home page HTML as seen in Figure 3

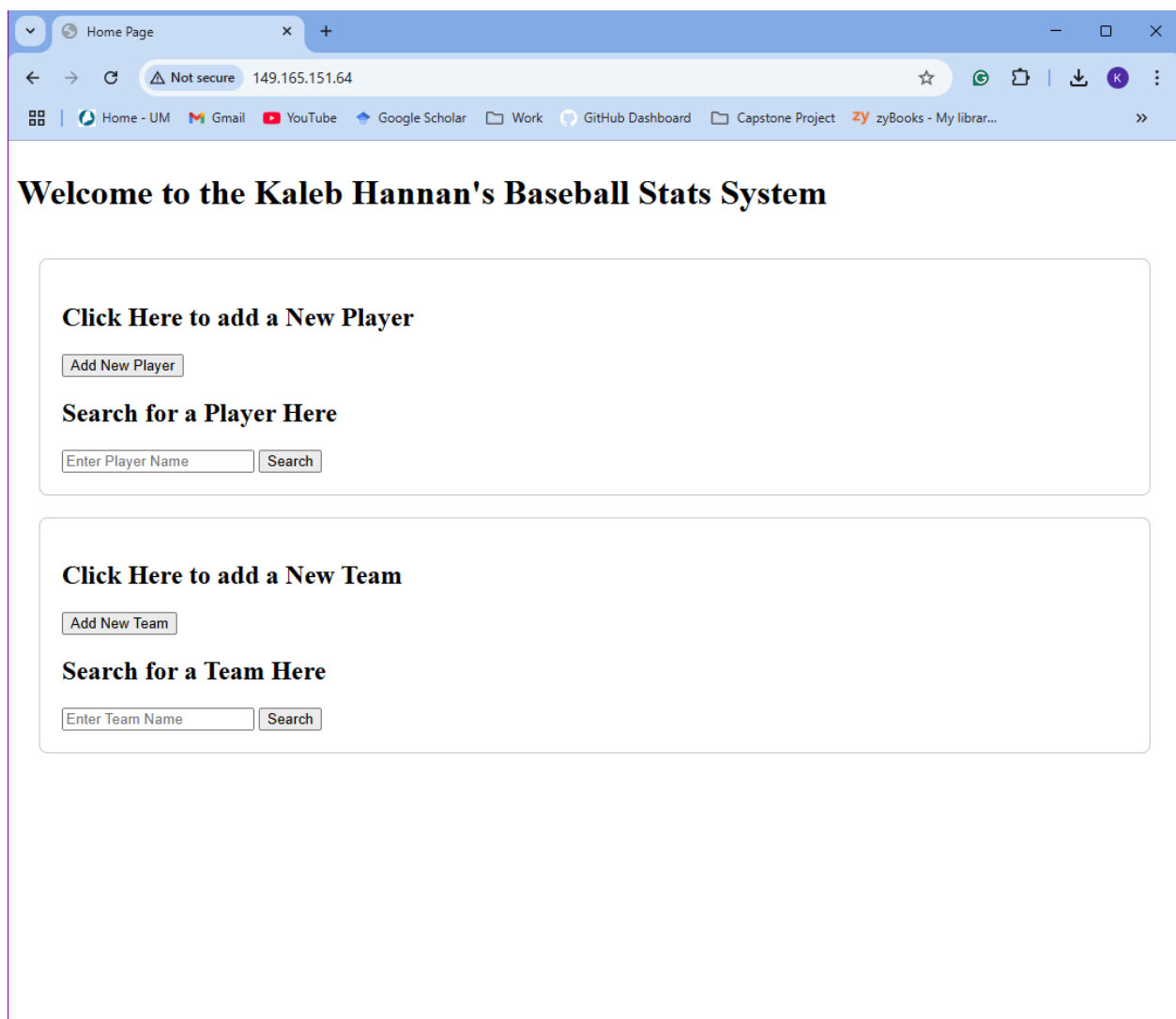


Figure 3: Home Page

```
@app.route('/searchForPlayer', methods=['POST'])
```

```
def searchForPlayer():
```

This route is called by a search form in the home.html file. It takes the player's name that was searched for on the home page and redirects the user to the viewPlayer route for the player searched for in the player-report container.

```
@app.route('/searchForTeam', methods=['POST'])
```

```
def searchForTeam():
```

This route is called by a search form in the home.html file. It takes the team's name that was searched for on the home page and redirects the user to the base route for the team searched in the team-report container.

Player-report:

This service is responsible for getting user input for players and sending that data to the player-bio, player-batting, and player-fielding-pitching containers. This service is also responsible for getting information from the player-bio, player-batting, and player-fielding-pitching containers and displaying the player's information in the browser. This service is also responsible for storing and retrieving the player's headshot from the Redis database.

```
@app.route('/viewPlayer/<player_name>')
```

```
def getplayer_report(player_name):
```

This route utilizes three GET requests to retrieve and display a player's information in the browser. It gets information from the player-bio container, the player-batting container, and the player-fielding-pitching container, and renders that information in an HTML file as shown in Figure 4.

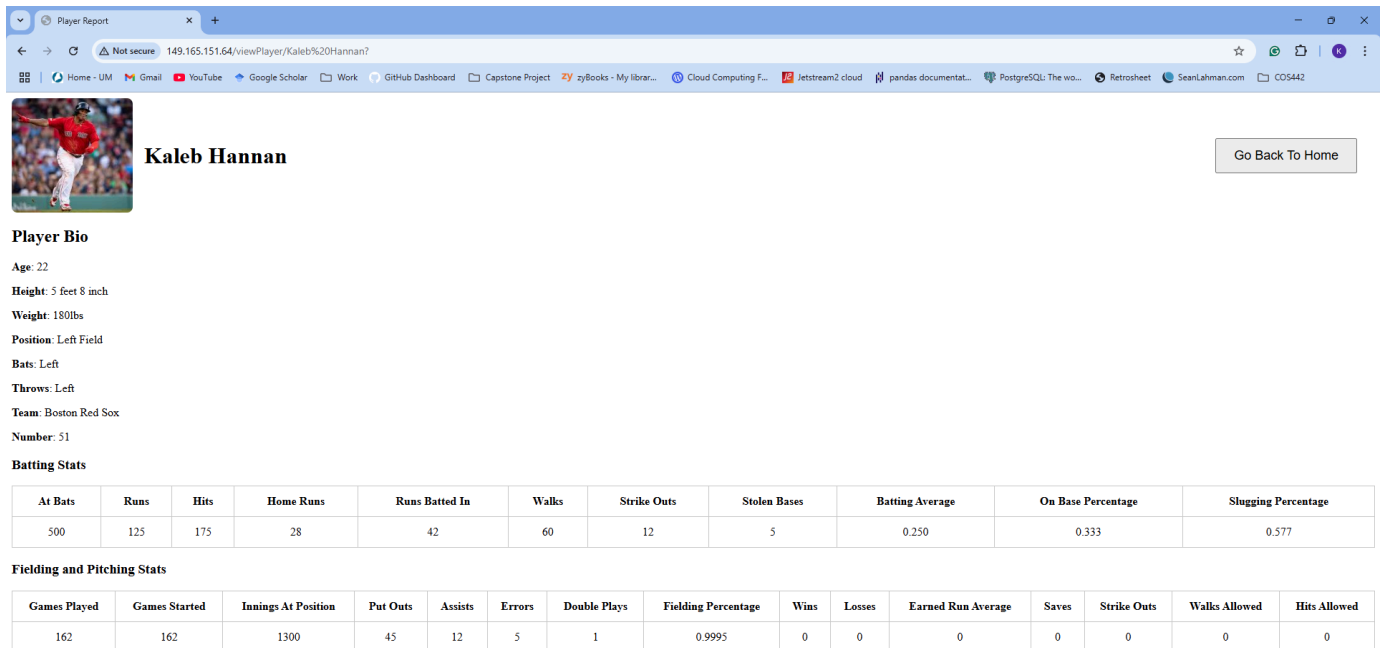


Figure 4: Player Report Page

```
@app.route('/', methods=['GET'])
```

```
def addNewPlayer():
```

This route displays a form to collect player bio information, as shown in Figure 5. The action for this form calls the /AddNewPlayerBatting route and uses a POST request to send the data from the form to the player-bio container, allowing it to store the data in the Redis database. This route also stores the player's headshot image in the Redis database.

Add New Player

Player Bio

Name:

Age:

Height:

Weight:

Position:

Team:

Number:

Bats:

Throws:

Upload Headshot:
 Devers.jff

Figure 5: Add new Player Bio Form

```
@app.route('/AddNewPlayerBatting', methods=['POST'])
```

```
def addNewPlayerBatting():
```

This route renders a form to collect players' batting information, as seen in Figure 6. The action for this form calls the /AddNewPlayerFieldingPitching route and uses a post request to send data from the form to the player-batting container, allowing it to store the data in the Redis database.

Batting Stats for Kaleb Hannan

At Bats:

Runs:

Hits:

Home Runs:

Runs Batted In:

Walks:

Strike Outs:

Stolen Bases:

Batting Average:

On Base Percentage:

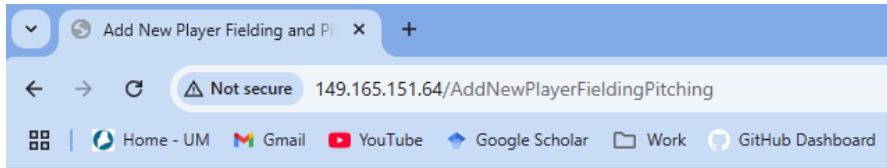
Slugging Percentage:

Figure 6: Add new Player batting stats form.

```
@app.route('/AddNewPlayerFieldingPitching', methods=['POST'])
```

```
def addNewPlayerFieldingPitching():
```

This route renders a form to collect players' fielding and pitching information, as seen in Figure 7. The action for this form calls the /AddNewPlayerComplete route and uses a post request to send data from the form to the player-fielding-pitching container, allowing it to store the data in the Redis database.



Fielding and Pitching Stats for Kaleb Hannan

Games Played:

Games Started:

Innings At Position:

Put Outs:

Assists:

Errors:

Double Plays:

Fielding Percentage:

Wins:

Losses:

Earned Run Average:

Saves:

Strike Outs:

Walks Allowed:

Hits Allowed:

Figure 7: Add New player fielding and pitching stats form

```
@app.route('/AddNewPlayerComplete', methods=['POST'])
```

```
def addNewPlayerComplete():
```

This route renders a confirmation screen after a new player has been added to the database. This screen allows the user to view the player's report or add a new player. This is seen in Figure 8.

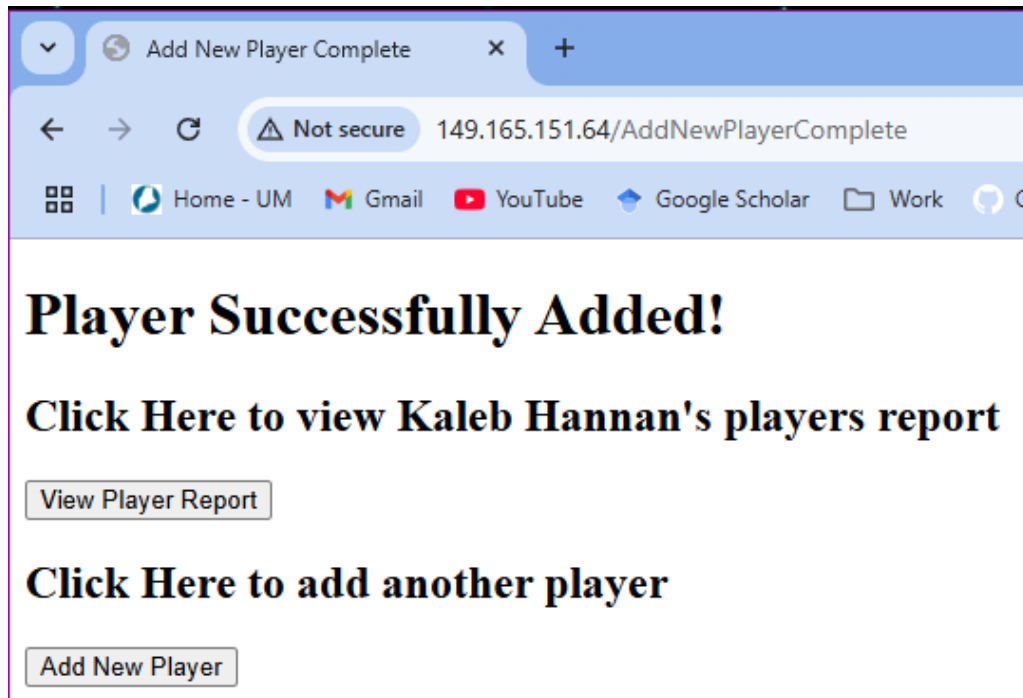


Figure 8: Add New Player Complete Page.

```
@app.route('/getHeadshot/<player_name>')
```

```
def getHeadshot(player_name):
```

This route is called in the viewPlayerHTML file. It gets the headshot for the player you are trying to view and sends it to the HTML template to display it on the browser, as seen in Figure 9.

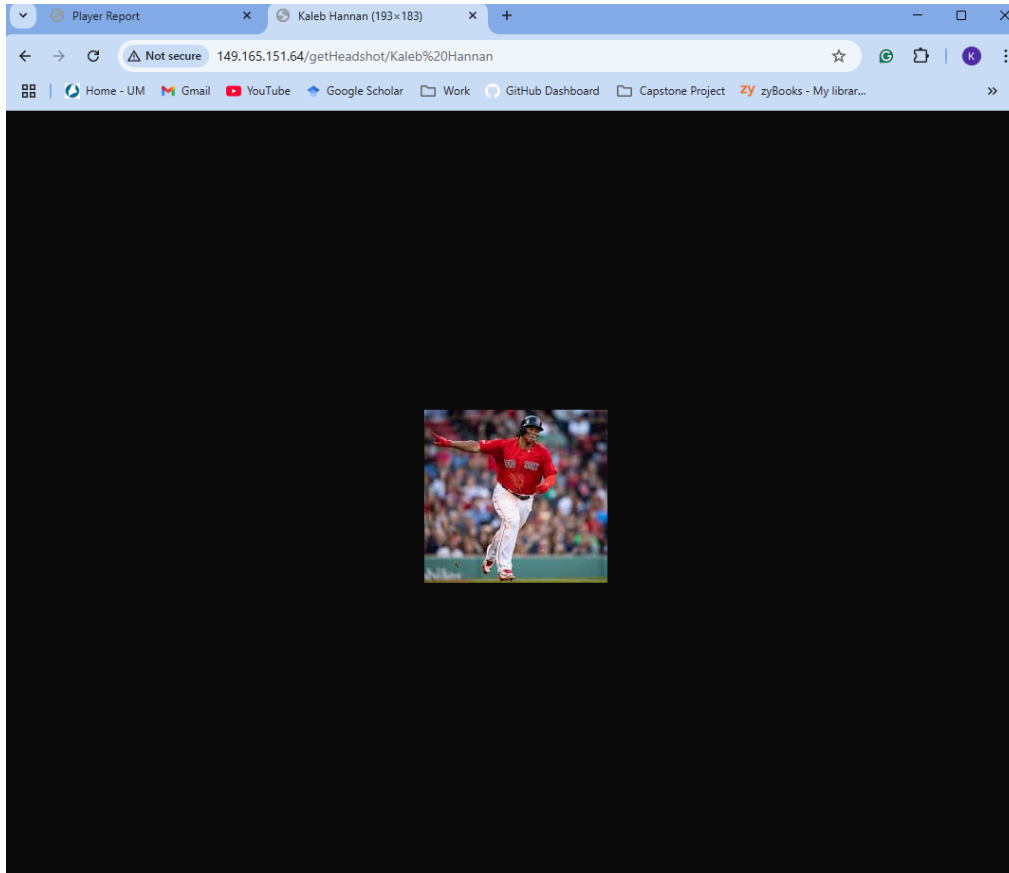


Figure 9: Get Headshot endpoint.

Player-bio:

The player bio service is responsible for storing and retrieving the player's Bio information from the Redis database.

@app.route('/getplayerBio/<player_name>')

def getplayerBio(player_name):

This route retrieves a player's Bio information from the Redis database. As seen in Figure 10.

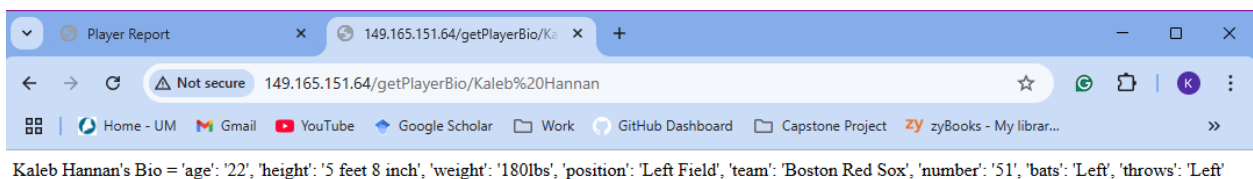


Figure 10: Get player bio information endpoint.

```
@app.route('/storeplayerBio', methods=['POST'])
```

```
def storeplayerBio():
```

This route stores a player's bio information in the Redis database and adds the player to a Redis set of players for the team that they are on.

Player-batting:

The player batting service is responsible for storing and retrieving the player's batting stats from the Redis database.

```
@app.route('/getplayerBatting/<player_name>')
```

```
def getplayerBatting(player_name):
```

This route retrieves a player's batting stats from the Redis database. As seen in Figure 11.

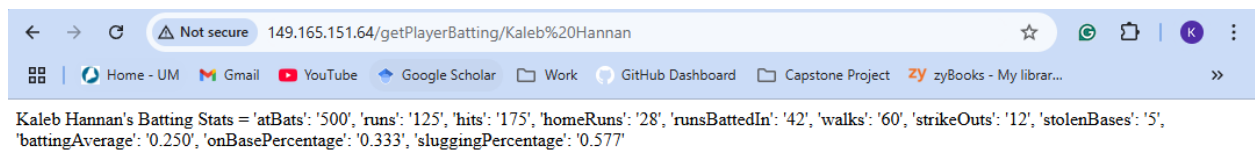


Figure 11: get player batting stats endpoint.

```
@app.route('/storeplayerBatting', methods=['POST'])
```

```
def storeplayerBatting():
```

This route stores a player's batting stats in the Redis database.

Player-fielding-pitching:

The player fielding-pitching service is responsible for storing and retrieving the player's fielding and pitching stats from the Redis database.

```
@app.route('/getplayerFieldingPitching/<player_name>')
```

```
def getplayerFieldingPitching(player_name):
```

This route retrieves a player's Fielding and Pitching stats from the Redis database. As seen in Figure 12.

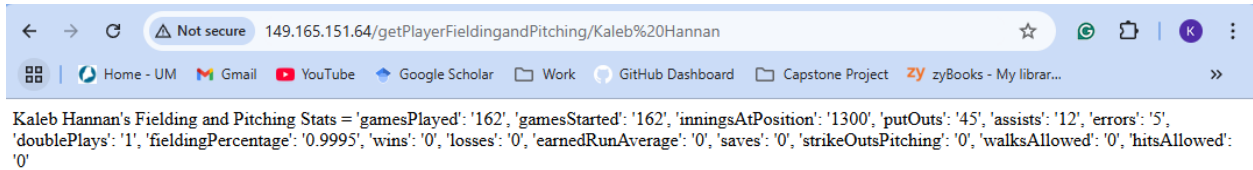


Figure 12: Get player Fielding and Pitching endpoint.

```
@app.route('/storeplayerFieldingPitching', methods=['POST'])
```

```
def storeplayerFieldingPitching():
```

This route stores players' fielding and pitching stats in the Redis database.

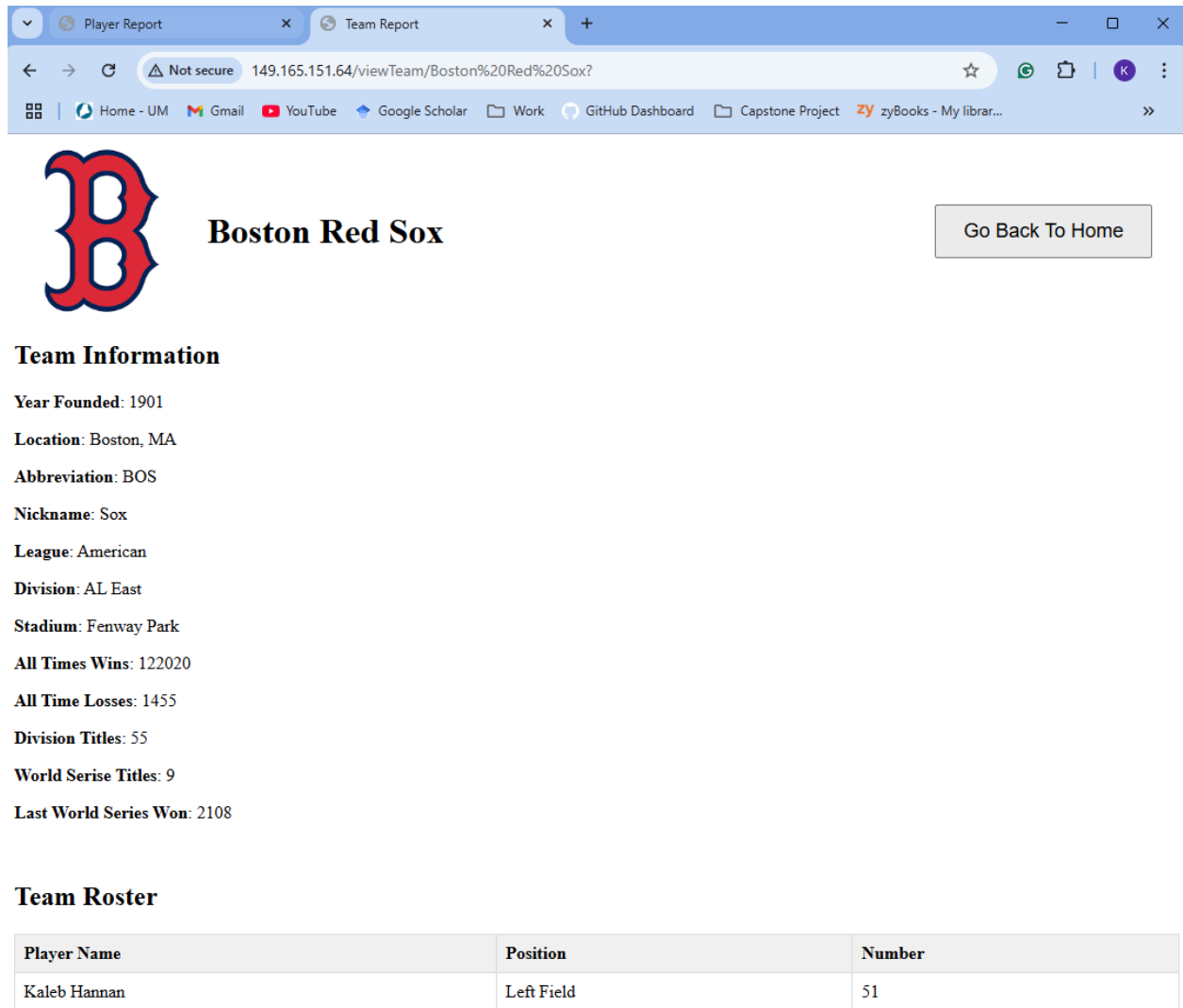
Team-report:

This service is responsible for getting user input for the teams added to the database and sending the information to the team-info service. These services also retrieve information from the team-info services and display the information in the browser. This service stores and retrieves the team logo images from the Redis database.

```
@app.route('/<team_name>')
```

```
def getTeamReport():
```

This route is responsible for rendering the HTML template to display the team information in the browser as seen in Figure 13. It utilized a GET request to get the team information from the team-info service. This route also gets the Redis Set of players associated with the team, and displays them on the Page.



Team Information

Year Founded: 1901

Location: Boston, MA

Abbreviation: BOS

Nickname: Sox

League: American

Division: AL East

Stadium: Fenway Park

All Times Wins: 122020

All Time Losses: 1455

Division Titles: 55

World Serise Titles: 9

Last World Series Won: 2108

Team Roster

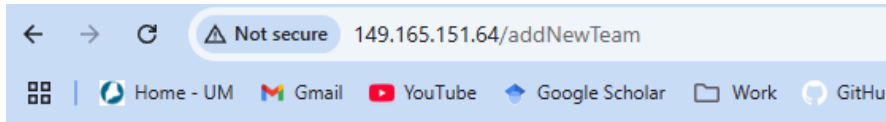
Player Name	Position	Number
Kaleb Hannan	Left Field	51

Figure 13: Team Report Page.

@app.route('/addTeamInfo')

def addNewTeam():

This route is responsible for rendering the HTML form for collecting the team's information, as seen in Figure 14.



Add New Team

Team Name:

Year Founded:

Location

Abbreviation

Nickname

League

Division

Stadium

All Time Wins

All Time Losses

Division Titles

World Series Titles

Last World Series Won

Upload Team Logo
 redsox.png

Figure 14: Add new team form.

```
@app.route('/addNewTeamComplete', methods=['POST'])
```

```
def addNewTeamComplete():
```

After adding a new team to the database, this route renders a confirmation screen. This screen allows the user to view the team's report or add a new team, as seen in Figure 15.

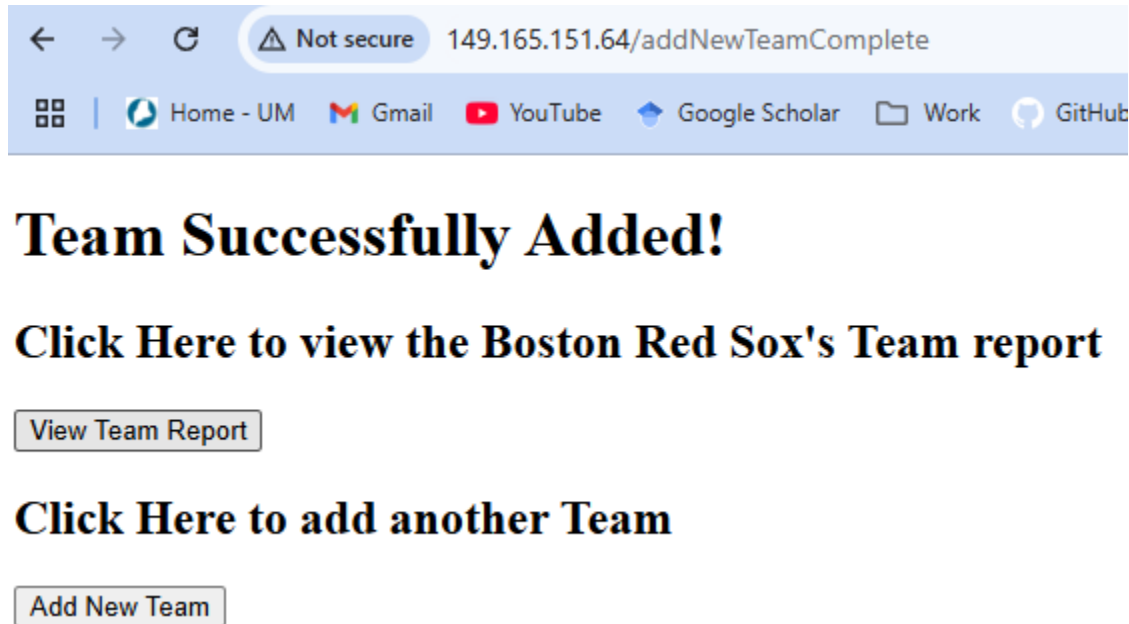


Figure 15: Add new team complete page.

```
@app.route('/getTeamLogo/<team_name>')
```

```
def getTeamLogo(team_name):
```

This route is called in the viewTeamInfo HTML file. It gets the team logo for the team you are trying to view and sends it to the HTML template to display on the browser as seen in Figure 16.

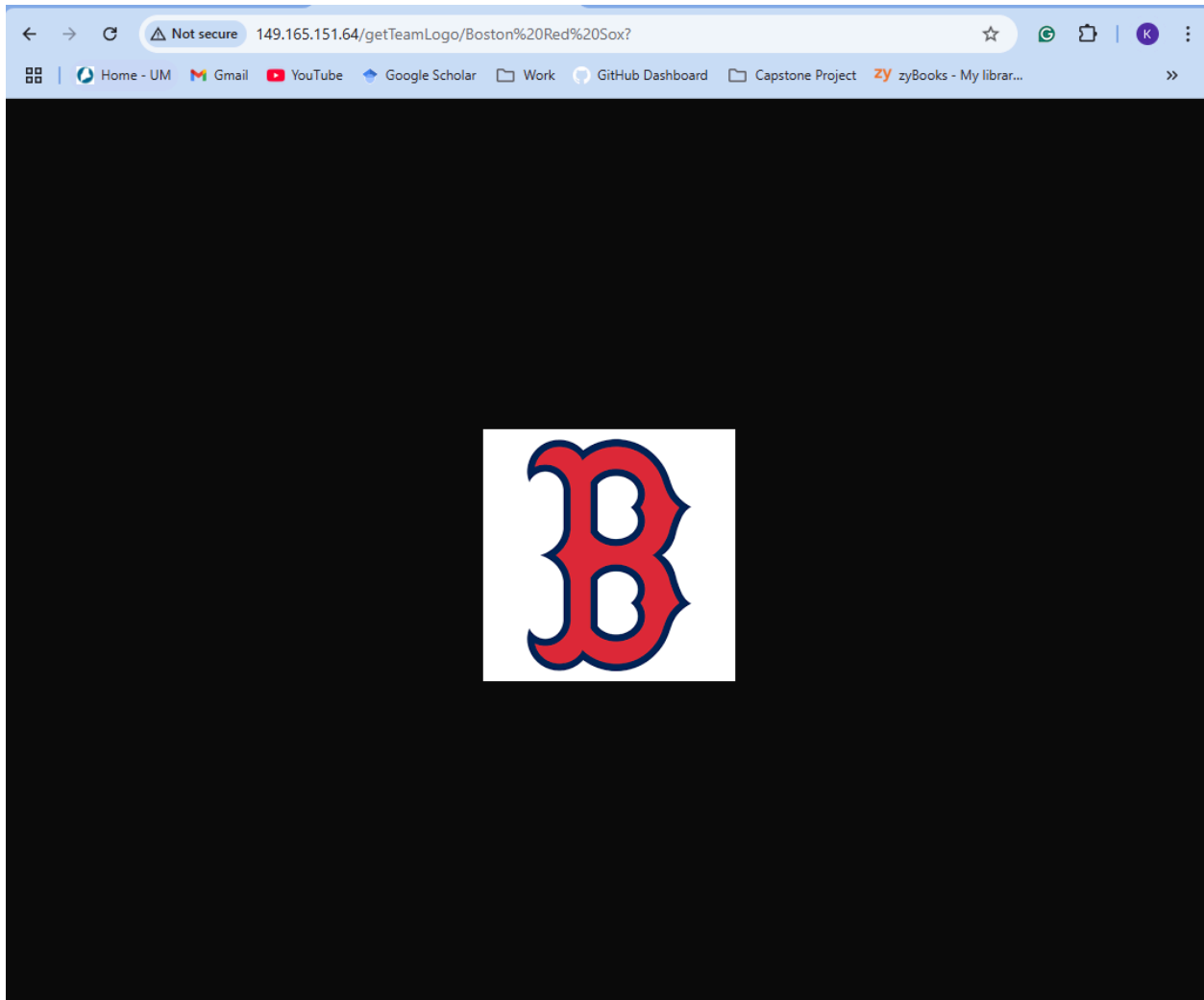


Figure 16: Get the team logo endpoint.

Team-Info:

@app.route('/getTeamInfo/<team_name>')

def getTeamInfo(team_name):

This route retrieves a team's information from the Redis database as seen in Figure 17.

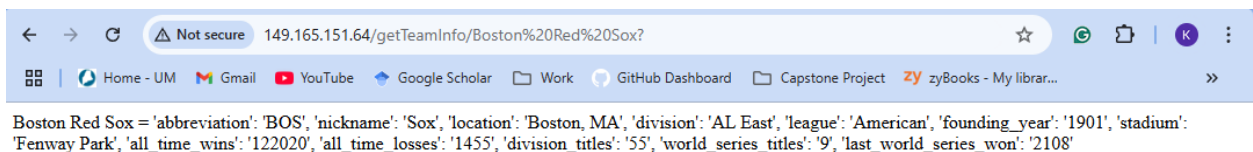


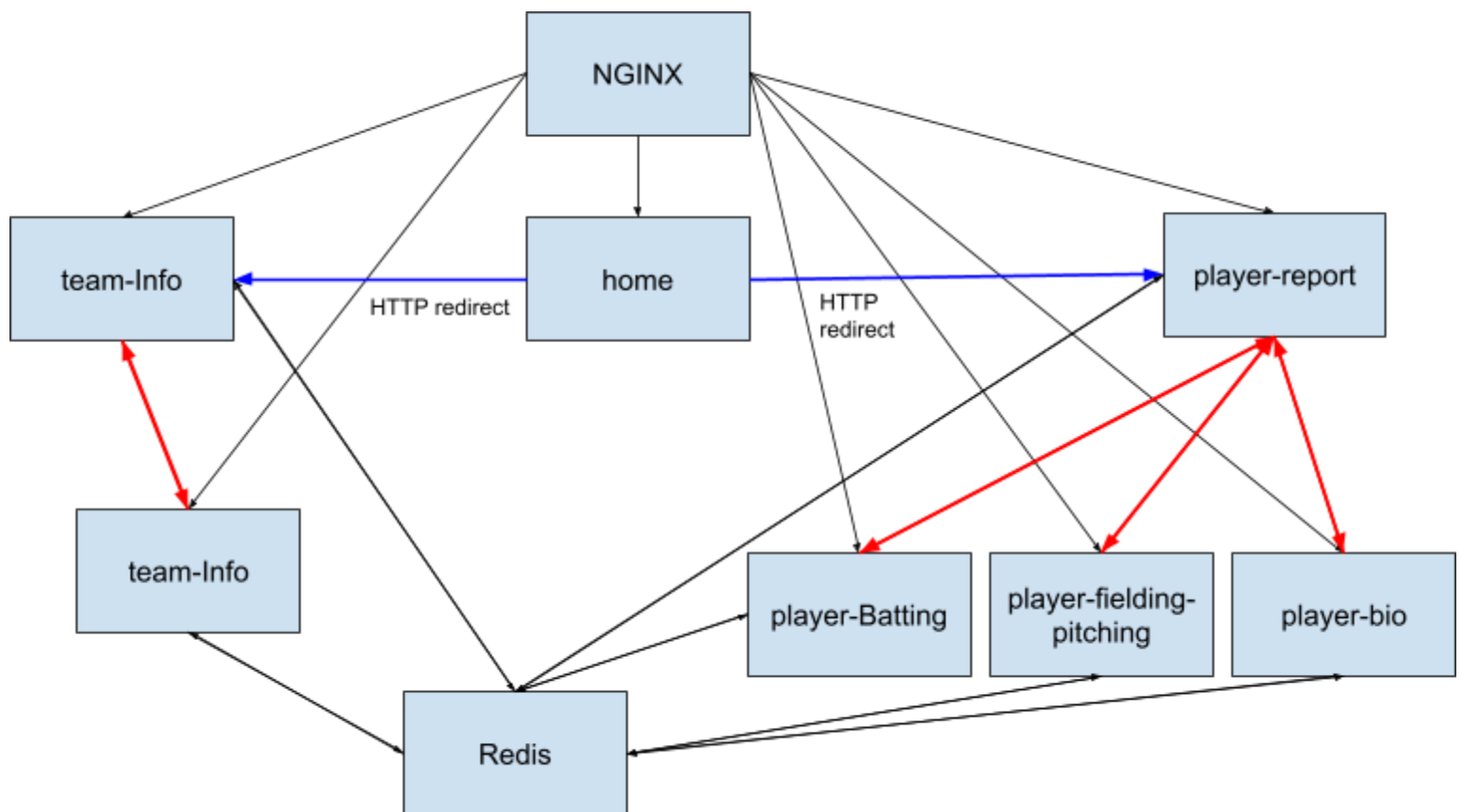
Figure 17: get team info endpoint

```
@app.route('/addTeamInfo')
```

```
def addTeamInfo():
```

This route stores a team's information in the Redis database.

Application Structure Diagram:



References for my Project:

- DataCamp. (2025, March 5). *Python Redis: A Beginner's Guide*.
<https://www.datacamp.com/tutorial/python-redis-beginner-guide>
- GeeksforGeeks. (2025, January 21). *Convert String Dictionary to Dictionary Python*.
<https://www.geeksforgeeks.org/python-convert-string-dictionary-to-dictionary/>
- GeeksforGeeks. (2025, April 26). *Redirecting to URL in Flask*.
<https://www.geeksforgeeks.org/redirecting-to-url-in-flask/>
- GeeksforGeeks. (2025, April 24). *StringIO and BytesIO for Managing Data as File Object*. GeeksforGeeks.
<https://www.geeksforgeeks.org/stringio-and-bytesio-for-managing-data-as-file-object/>
- Pallets Projects. (n.d.). *API — Flask Documentation (3.1.x)*. Flask.
https://flask.palletsprojects.com/en/stable/api/#flask.send_file
- Pallets Projects. (n.d.). *Templates — Flask Documentation (3.1.x)*. Flask.
<https://flask.palletsprojects.com/en/stable/templating/>
- Redis. (n.d.). *Redis sets*. Redis Documentation.
<https://redis.io/docs/latest/develop/data-types/sets/>
- Stack Overflow. (2019, July 9). *How to send an image directly from flask server to html?*
<https://stackoverflow.com/questions/56946969/how-to-send-an-image-directly-from-flask-server-to-html>
- W3Schools. (n.d.). *How To Create a File Upload Button*. W3Schools.
https://www.w3schools.com/howto/howto_html_file_upload_button.asp
- W3Schools. (n.d.). *HTML Forms*. W3Schools.
https://www.w3schools.com/html/html_forms.asp
- W3Schools. (n.d.). *HTML Tables*. W3Schools.
https://www.w3schools.com/html/html_tables.asp
- W3Schools. (n.d.). *HTML <select> Tag*. W3Schools.
https://www.w3schools.com/tags/tag_select.asp